

CSP初赛集训
第9课
各类算法

刘渠

2020.07.20

目录/CONTENTS

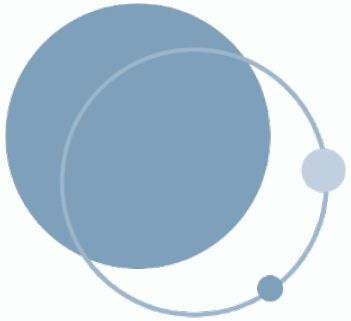
01 搜索算法

02 递归递推

03 分治算法

04 贪心算法

05 动规算法



01 搜索算法



枚举算法

枚举，也称“暴力搜索”，就是将所有情况都列举出，并判断该情况是否符合题目条件。

- 方法步骤：

1.确定枚举对象、枚举范围、判断条件。

2.循环验证每一个解。

- 适用于数据规模较小的问题

枚举思想的应用

- 求最值
- 选择排序
- 顺序查找
- 求不定方程的解
- 求字符串的所有子串

同时求最大最小值

[J-2018-9]给定一个含N 个不相同数字的数组，在最坏情况下，找出其中最大或最小的 数，至少需要 N - 1 次比较操作。则最坏情况下，在该数组中同时找最大与 最小的数至少需要（ ）次比较操作。（□ □ 表示向上取整，□ □ 表示向下取整)

A. □ 3N / 2□ - 2

B. □ 3N / 2□ - 2

C. 2N - 2

D. 2N - 4

正确答案： A

深度优先搜索算法

深度优先搜索算法（DFS)，简称**深搜**，或**回溯法**、试探法。

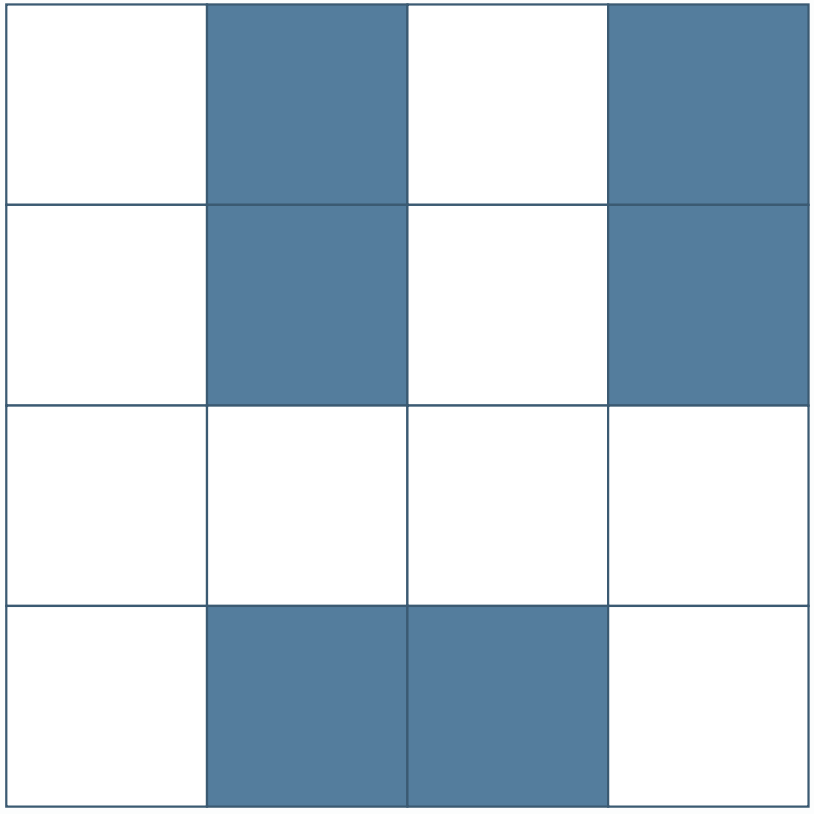
在包含问题的所有解的解空间树中，按照深度优先的策略，从根结点出发探索解空间树。

深度优先策略：从一条路走到底，能进则进，不能进则退回来。

深搜可以解决：求所有路径数，所有方案数问题。

深度优先搜索求迷宫问题

起点



终点

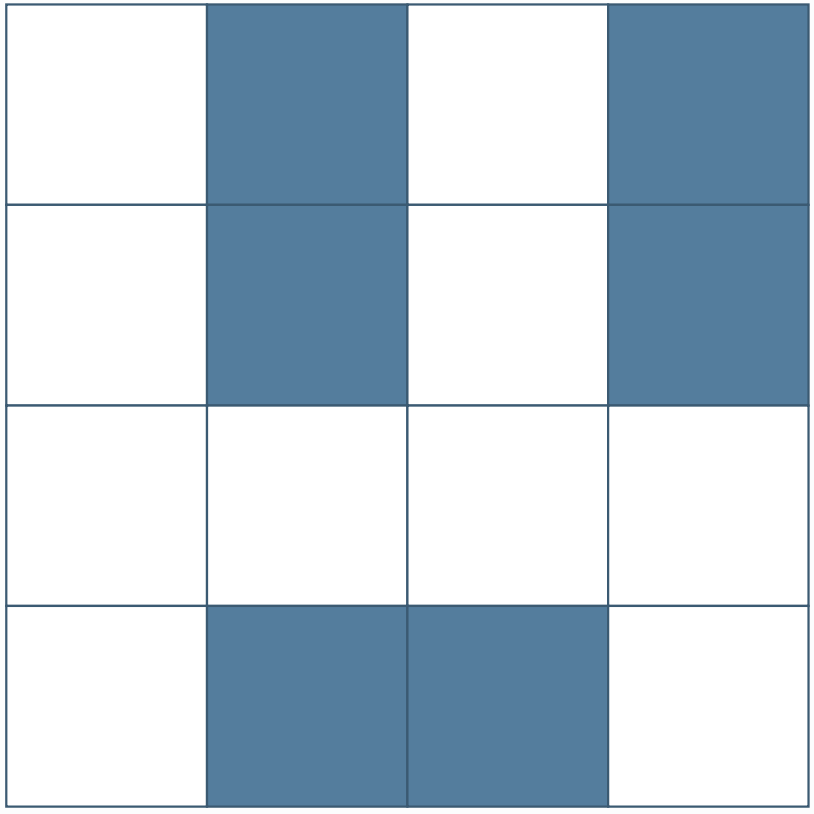
广度优先搜索算法

广度优先搜索（Breadth First Search，简称BFS），
又称宽度优先搜索，简称：广搜、宽搜。
在解空间树上，从初始结点开始，逐层遍历结点，
直到找到解。

广搜常用于解决求最短路径，求最少步数的问题。
广搜中需要用到队列

广度优先搜索求迷宫问题

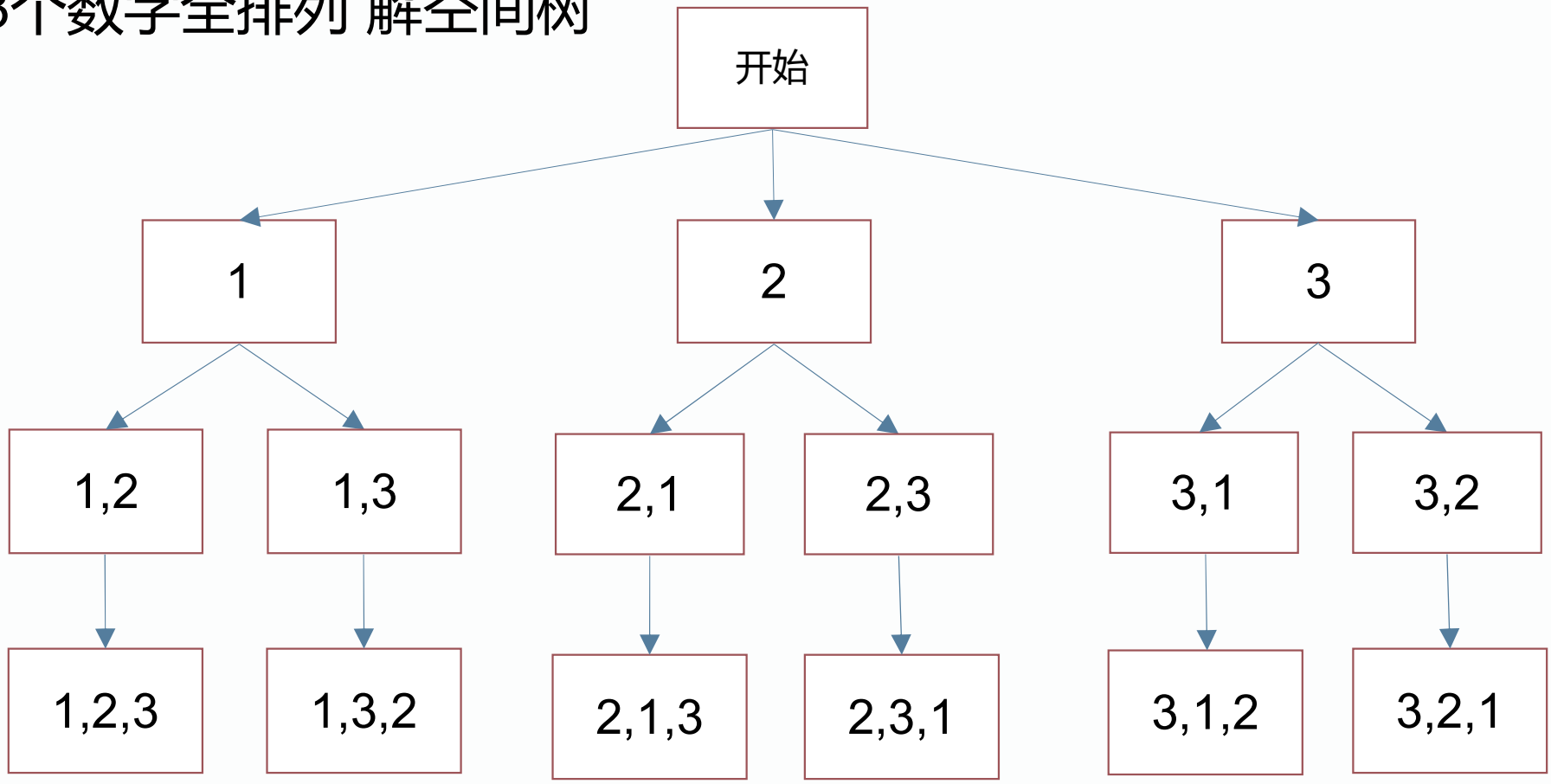
起点



终点

解空间树

3个数字全排列 解空间树



练习

[J-2011-11][S-2011-5]广度优先搜索时，需要用到的数据结构是（ ）。

A. 链表

B. 队列

C. 栈

D. 散列表

正确答案：B

02 递推与递归



递推

通过已知的初始状态及状态间的关系，逐步推出要得到的问题结果。

例：求1+2+...+n的和

```
int n, s[100];
cin>>n;
s[1] = 1;
for(int i = 2; i <= n; ++i)
    s[i] = s[i-1] + i;
cout<<s[n];
```

递归

将大规模的问题不断分解为小规模的问题，直至问题可以直接解决。

例：求1+2+...+n的和

```
int sum(int n)
{
    if(n == 1)
        return 1;
    else
        return n + sum(n-1);
}
```

栈溢出

如果**没有**设置递归出口（递归的结束条件），函数栈帧不断入栈，占用空间会超出栈区空间，导致**栈溢出**。

栈溢出会使有用的存储单元被改写，往往会引发不可预料的后果。



递推与递归的相似点

- 都可以解决结构自相似的问题

要解决一个较大规模的问题，都要依托同类较小规模问题的解。

递推

1. 确定**递推状态**
2. 确定**初始状态**
3. 确定**递推关系**

递归

1. 确定**递归问题**
2. 确定**递归关系**
3. 确定**递归出口**

递推与递归的不同点

- 考察对象不同

递推

递归

递推状态：

问题的结果

递归问题：

问题本身

递推与递归的不同点

- 思维方式不同

递推

从初始状态出发，
从前向后推，
直到得到最终结果。

递归

从最终问题出发，
从后向前推，
直到找到可以直接解决的基本问题。

递推与递归的不同点

- 复杂度略微不同

  递推与记忆化递归的复杂度在相同的数量级

递推

时间复杂度：较低

空间复杂度：较低

递归

时间复杂度：较高

空间复杂度：较高

递推与递归的不同点

- 可以做的事情

递推

一般只能求数值

例：

求汉诺塔的移动次数

递归

可以做复杂的过程

例：

求汉诺塔的移动过程

练习

[J-2020-6]设A是由实数组成的数组，考虑下面的递归算法：

```
XYZ (A[1..n])
1.  if n= 1 then return A[1]
2.   else temp ← XYZ (A[1...n-1])
3.         if temp < A[n]
4.         then return temp
5.         else return A[n]
```

请问算法XYZ的输出是什么？（）。

A. A数组的平均值　　B. A数组的最小值

C. A数组的中值　　D. A数组的最大值

正确答案：B

03 分治算法



分治算法

分治算法：将一个难以直接解决的大问题，分割成一些规模较小的相同问题，通过合并小规模问题的解得到整个问题的解。

分治算法的应用：

- 分治求数组最值
- 归并排序
- 快速排序
- 二分查找与二分答案

练习

[J-2016-14][S-2016-15]给定含有 n 个不同的数的数组L=<x1,x2,...,xn>。如果L中存在xi(1<i<n)使得x1<x2<...<xi-1<xi>xi+1>...>xn， 则称L是单峰的，并称xi 是L的“峰顶”。现在已知 L 是单峰的,请把 a-c 三行代码补全到算法中使得算法 正确找到 L 的峰顶。

a. Search(k+1, n)
b. Search(1, k-1)
c. return L[k]

```
Search(1, n)
1. k ← floor(n/2)
2. if L[k] > L[k-1] and L[k] > L[k+1]
3. then _____
4. else if L[k] > L[k-1] and L[k] < L[k+1]
5. then _____
6. else _____
```

正确的填空顺序是（）。

A. c,a,b　B. c,b,a　C. a,b,c　D. b,a,c

正确答案： A

二分查找

二分查找（也叫折半查找），是一种在**有序序列**上的查找算法。
查找思路：

●对于有序数列（从小到大），设定左端l（最小元素下标）和右端r（最大元素下标）

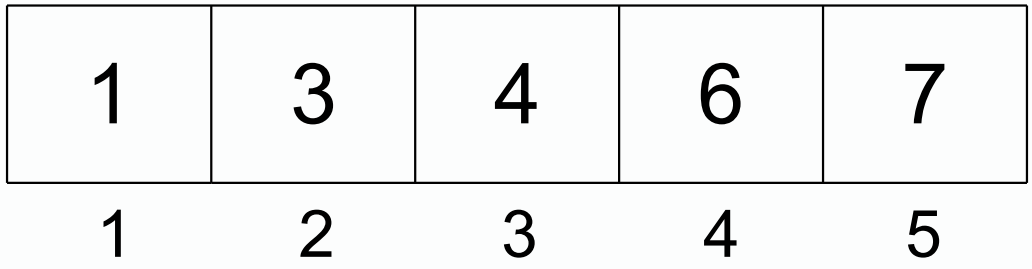
●当满足条件l <= r时，求中点m=(l+r)/2，将中点元素的值与所要查找的值比较

- 若中点元素值比所要查找元素小，则应找后半段，所以l = m+1。
- 否则应找前半段r = m - 1，直到找到为止。
- 若l > r，则说明找不到。

二分查找

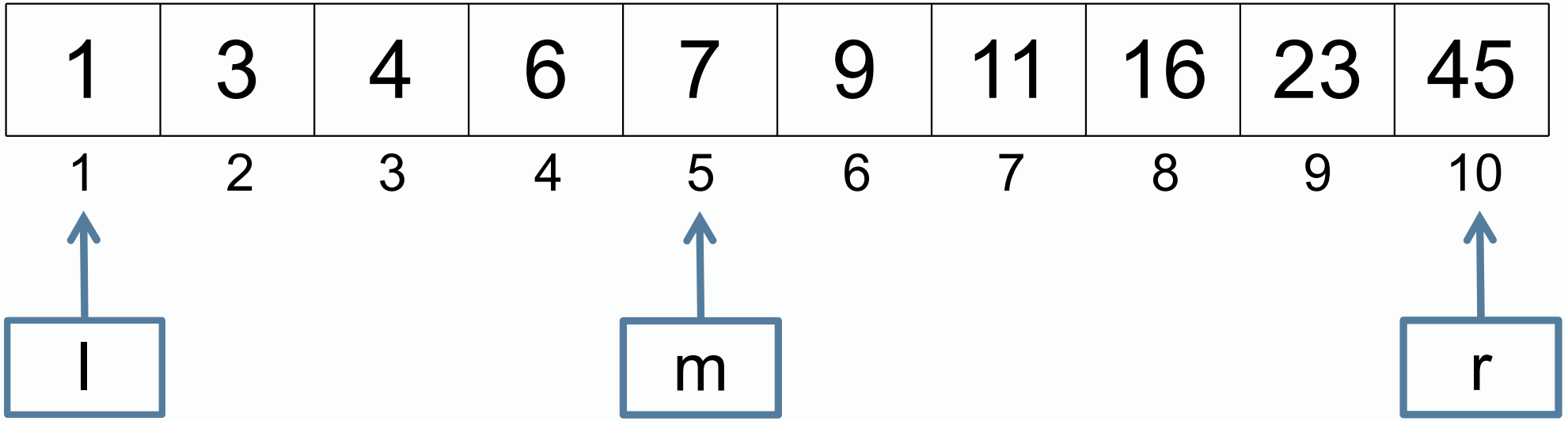
升序数组，各元素不同，查找某元素。

- 如果该元素存在：输出该元素的下标
- 如果不存在该元素，输出-1



二分查找

在数字序列中1 3 4 6 7 9 11 16 23 45，查找11的位置

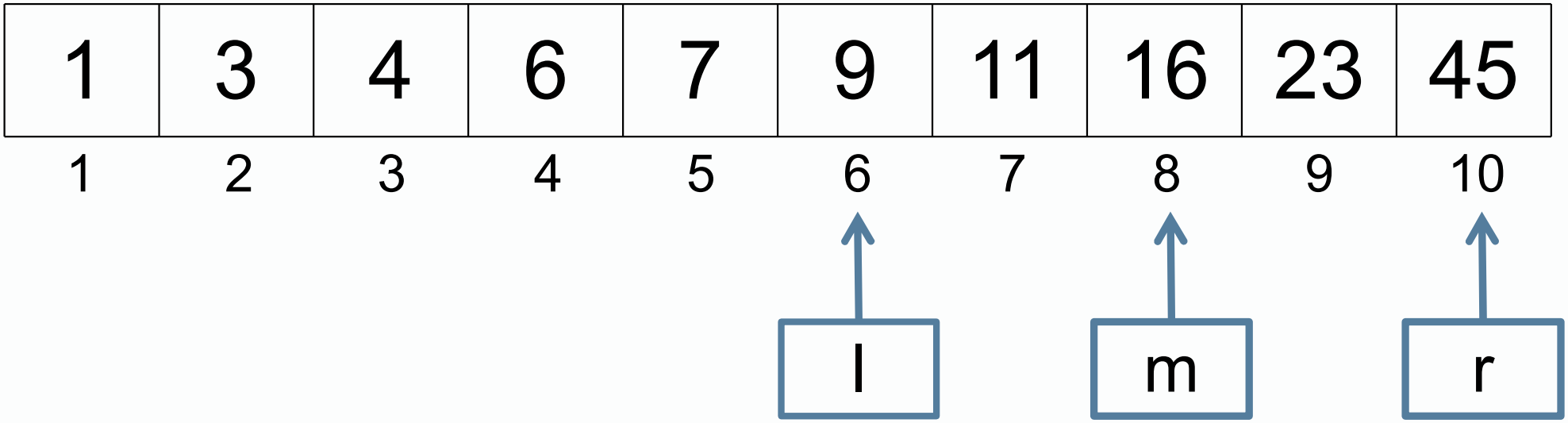


m = (l + r) / 2

l = m + 1

二分查找

查找11的位置



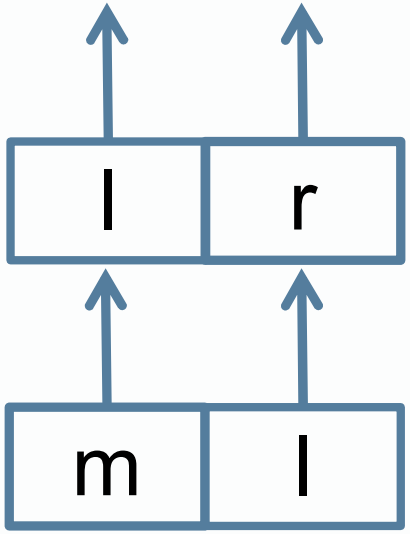
m = (l + r) / 2

r = m - 1

二分查找

查找11的位置

1	3	4	6	7	9	11	16	23	45
1	2	3	4	5	6	7	8	9	10

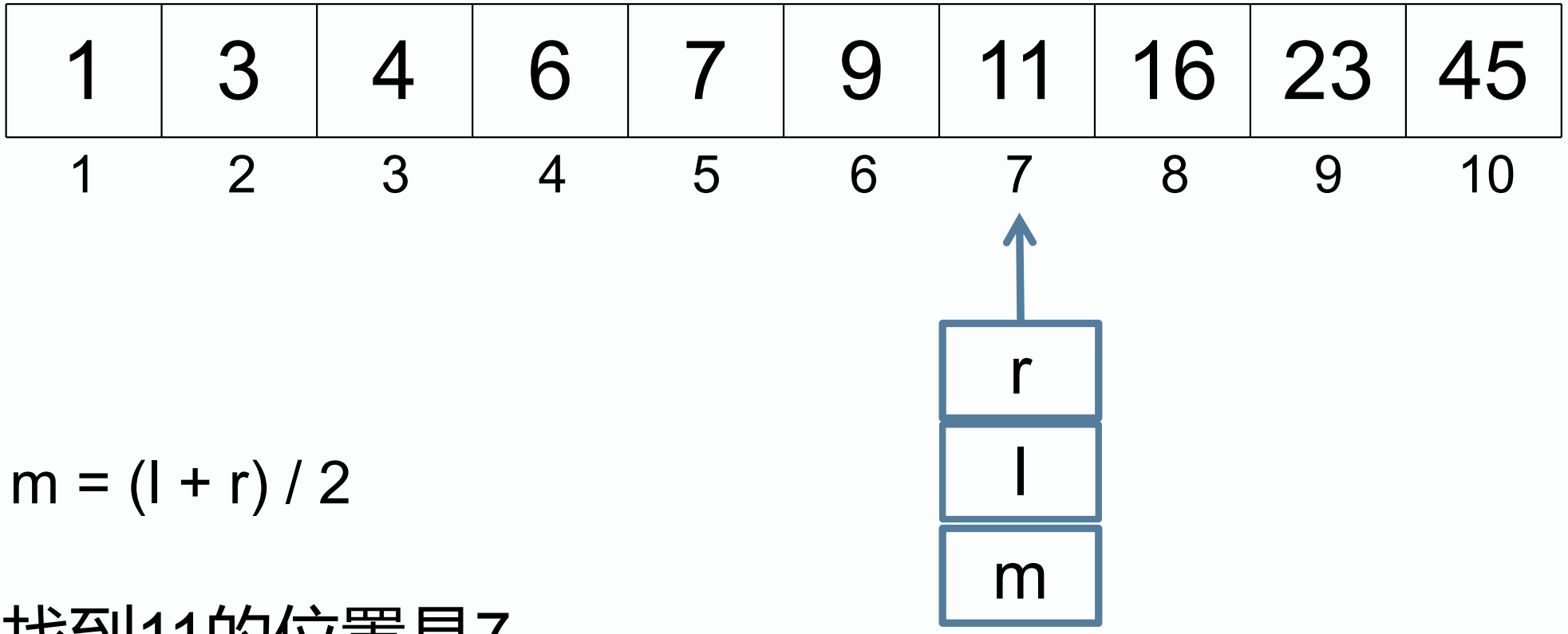


m = (l + r) / 2

l = m + 1

二分查找

查找11的位置

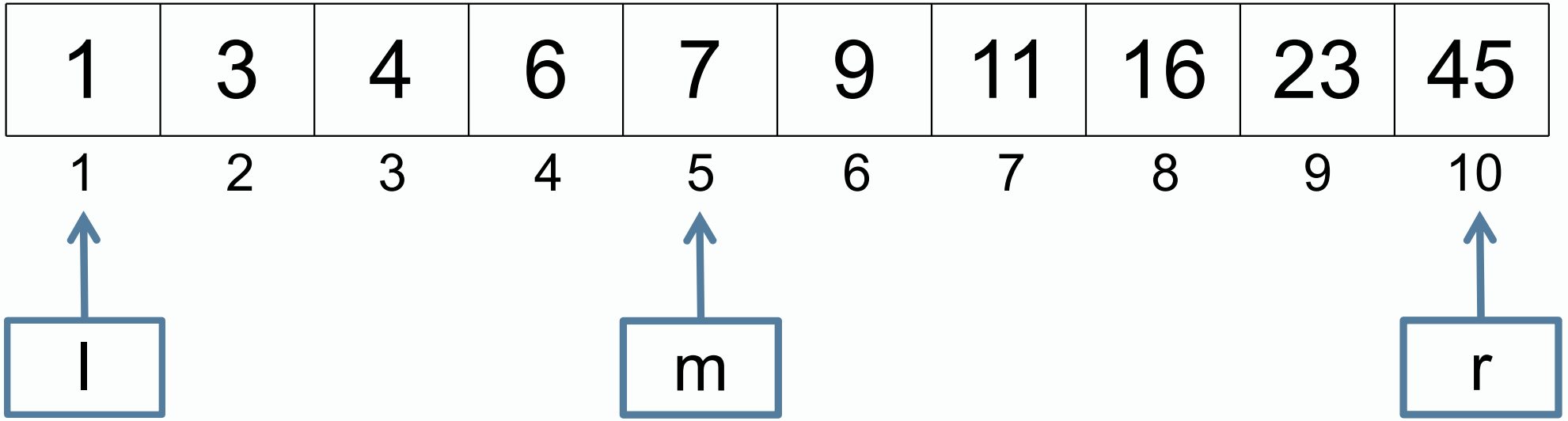


m = (l + r) / 2

找到11的位置是7

二分查找

查找10的位置

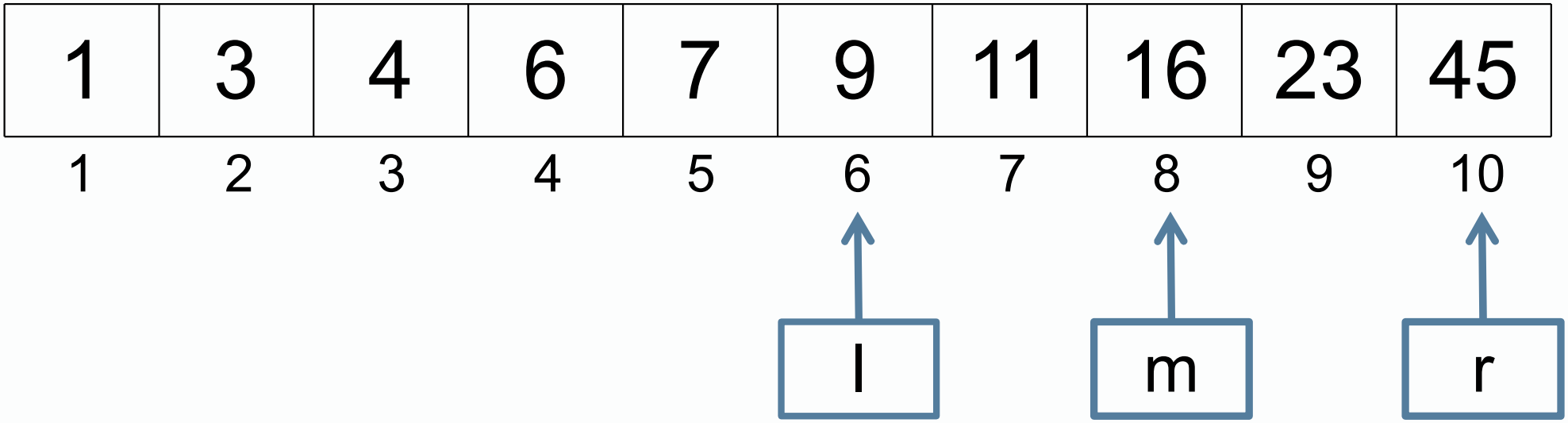


m = (l + r) / 2

l = m + 1

二分查找

查找10的位置



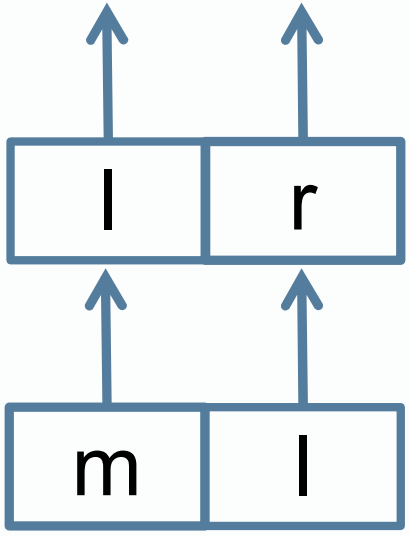
m = (l + r) / 2

r = m - 1

二分查找

查找10的位置

1	3	4	6	7	9	11	16	23	45
1	2	3	4	5	6	7	8	9	10



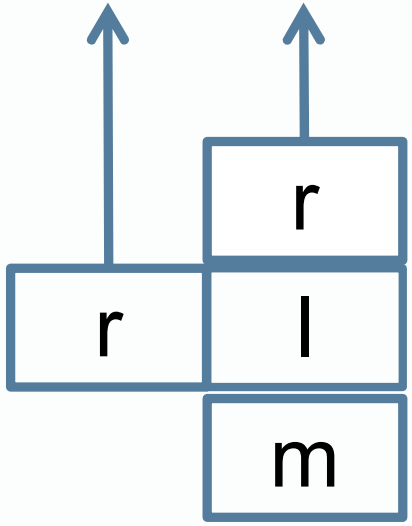
m = (l + r) / 2

l = m + 1

二分查找

查找10的位置

1	3	4	6	7	9	11	16	23	45
1	2	3	4	5	6	7	8	9	10



m = (l + r) / 2

r = m - 1

不再满足l <= r，查找结束，没有找到

二分查找

```
//输入n，输入n个数（升序，各不相同），输入x，求x是第几个数
#include <bits/stdc++.h>
using namespace std;
int main()
{
    int n, a[1005], x, l, r, m;
    cin >> n;
    for(int i = 1; i <= n; ++i)
        cin >> a[i];
    cin >> x;
    l = 1, r = n;
    while(l <= r)//l在r左边
    {
        m = (l + r) / 2;//取中点位置
        if(x > a[m])//如果x在右半部分
            l = m + 1;//l移到中点右边
        else if(x < a[m])//如果x在左半部分
            r = m - 1;//r移到中点左边
        else
        {
            cout << m;//如果中点是要查找的数字，那么输出该位置
            return 0;
        }
    }
    cout << -1;//没找到，输出-1
    return 0;
}
```

二分查找

在n个元素的有序序列中进行二分查找

- 最大比较次数：$\lfloor log_2 n \rfloor + 1$
- 时间复杂度：O(logn)

练习

[J-2009-16]有一个由4000个整数构成的顺序表，假定表中的元素已经按升序排列，采用二分查找定位一个元素。则最多需要几次比较就能确定是否存在所查找的元素：

A. 11次

B. 12次

C. 13次

D. 14次

正确答案： B

04 贪心算法



贪心算法

贪心算法是指在求解问题时，总是做出某种意义上的局部最优选择（贪心选择），而不是考虑整体的最优解。

例：凑n元钱，看哪种方案使用的纸币最少。每次选择可以选择的面值最大的纸币。

贪心选择性质：一个问题的整体最优解可通过一系列的**贪心选择**达到。

贪心算法

贪心算法的应用:

- 部分背包问题
- 求哈夫曼树
- 求最小生成树Prim算法
- 求最小生成树Kruskal算法
- 求最短路Dijkstra算法

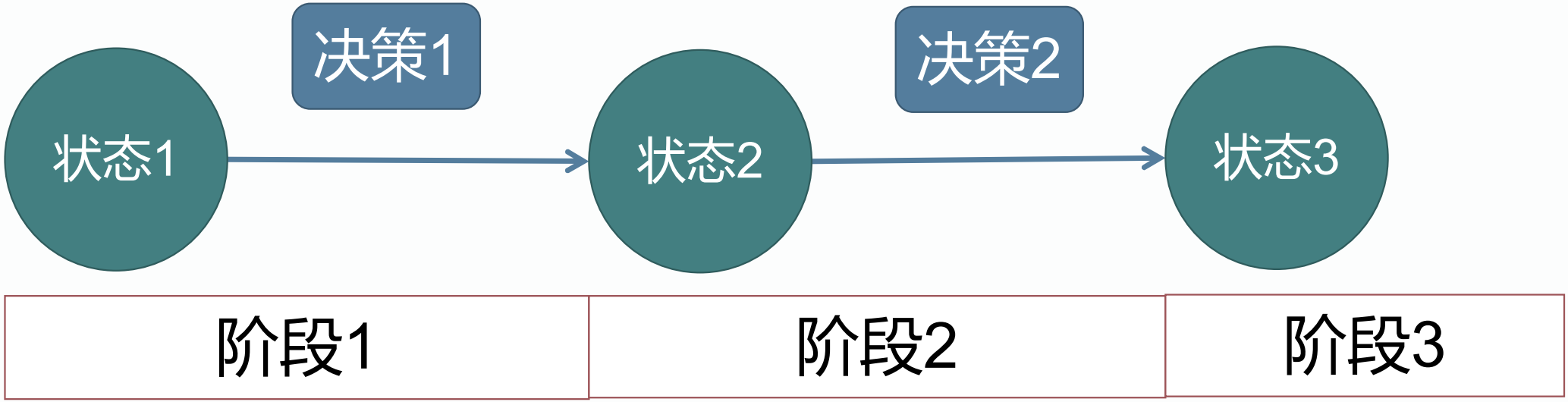
练习

[J-2019-11]新学期开学了，小胖想减肥，健身教练给小胖制定了两个训练方案。
方案一：每次连续跑3公里可以消耗300千卡（耗时半小时）；
方案二：每次连续跑5公里可以消耗600千卡（耗时1小时）。
小胖每周周一到周四能抽出半小时跑步，周五到周日能抽出一小时跑步。
另外，教练建议小胖每周最多跑21公里，否则会损伤膝盖。
请问如果小胖想严格执行教练的训练方案，并且不想损伤膝盖，每周最多通过跑步消耗多少千卡？（）
A. 3000　B. 2500　C. 2400　D. 2520
正确答案：C

05 动态规划算法



多阶段决策过程



多阶段决策过程: 问题可以按时间顺序分解成若干相互联系的阶段，在每一个阶段都要做出决策，全部过程的决策是一个**策略**。

动态规划可以解决的问题

动态规划方法可以解决多阶段决策过程最优化问题。问题需满足以下性质：

- 多阶段决策过程
- 最优子结构
- 无后效性
- 包含重叠子问题

通常使用递推求解

动态规划相关概念

- 阶段，阶段变量
- 决策，决策变量，允许的决策集合
- 策略，策略集合，最优策略
- 状态，状态变量

在策略集合中选择满足某种条件子集的特征值。

- 状态转移方程

分析方法：分割策略集合

动态规划问题分类

- 坐标型dp
- 线性dp
- 区间dp
- 背包dp
- 树形dp
- 图上dp

- 滚动数组优化
- 二进制优化
- 状态压缩优化
- 单调队列优化
- 斜率优化

练习

[S-2021-13]有 8 个苹果从左到右排成一排，你要从中挑选至少一个苹果，并且不能同时挑选相邻的两个苹果，一共有（ ）种方案。

A. 36

B. 48

C. 54

D. 64

正确答案： C

06 排序算法



排序算法稳定性

如果相等的元素并不相同　　2 3 2 1

冒泡排序：

2 3 2 1

2 2 1 3

2 1 2 3

1 2 2 3

直接选择排序：

2 3 2 1

1 3 2 2

1 2 3 2

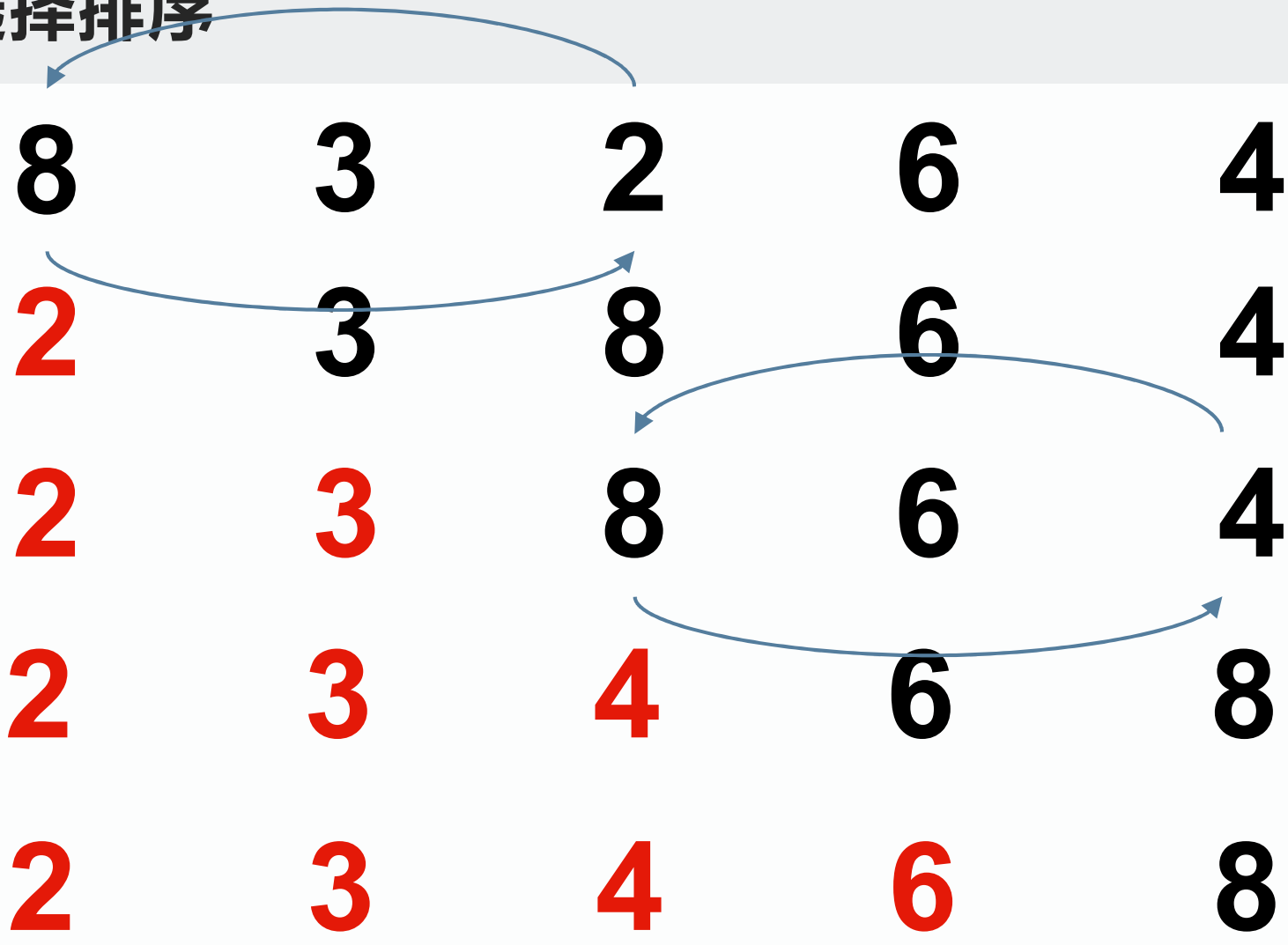
1 2 2 3

排序算法的稳定性

假定在待排序的记录序列中，存在多个具有相同的关键字的记录，若经过排序，这些记录的相对次序保持不变，则称这种排序算法是稳定的；否则称为不稳定的。

- 稳定的排序可以利用上一次的排序结果，来服务这次的排序
- 判断方法：排序时是否只有相邻元素交换

选择排序



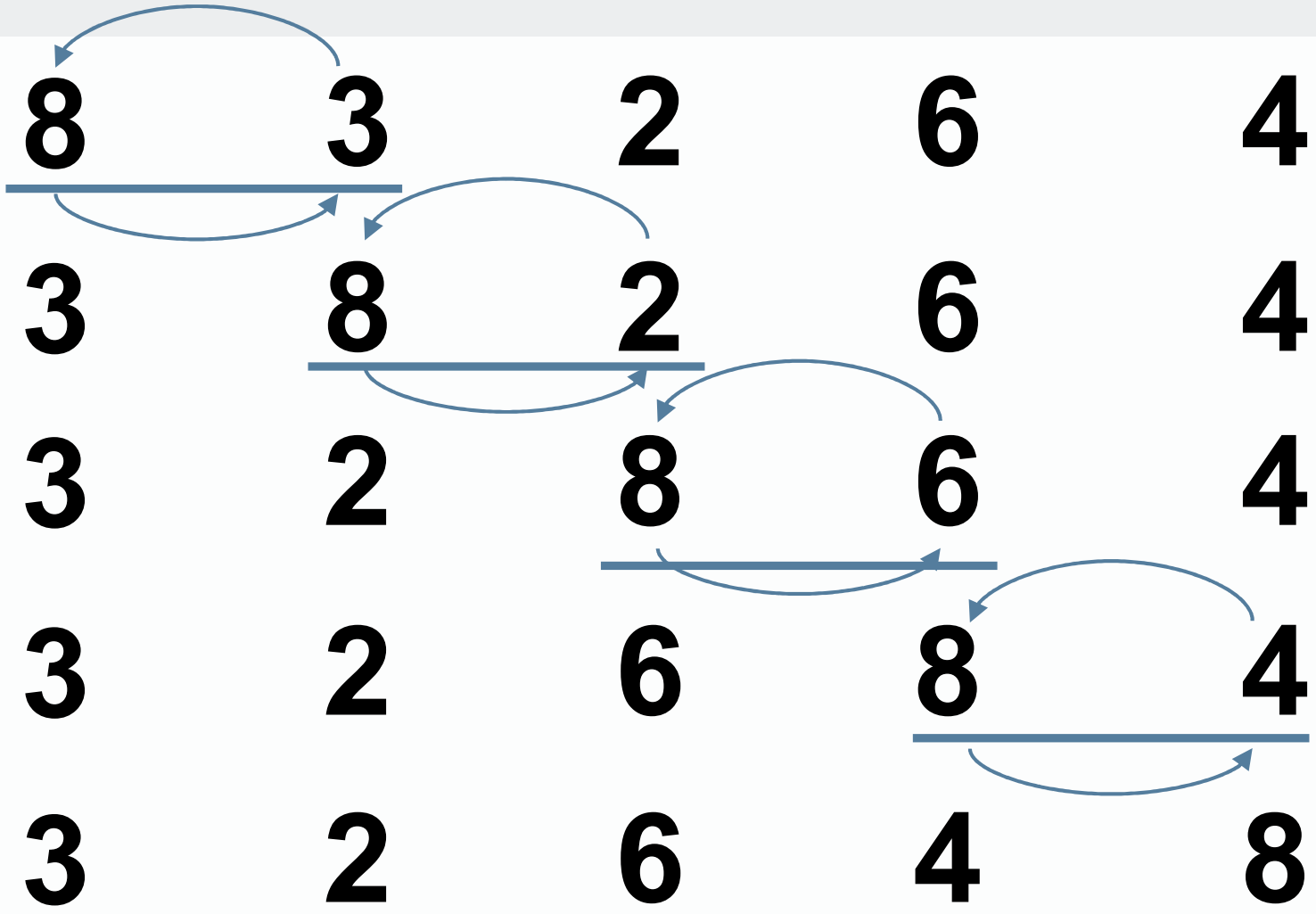
选择排序

选择排序：每一趟从待排序的数据元素中选出最小（或最大）的一个元素，顺序放在待排序的数列的最前面。

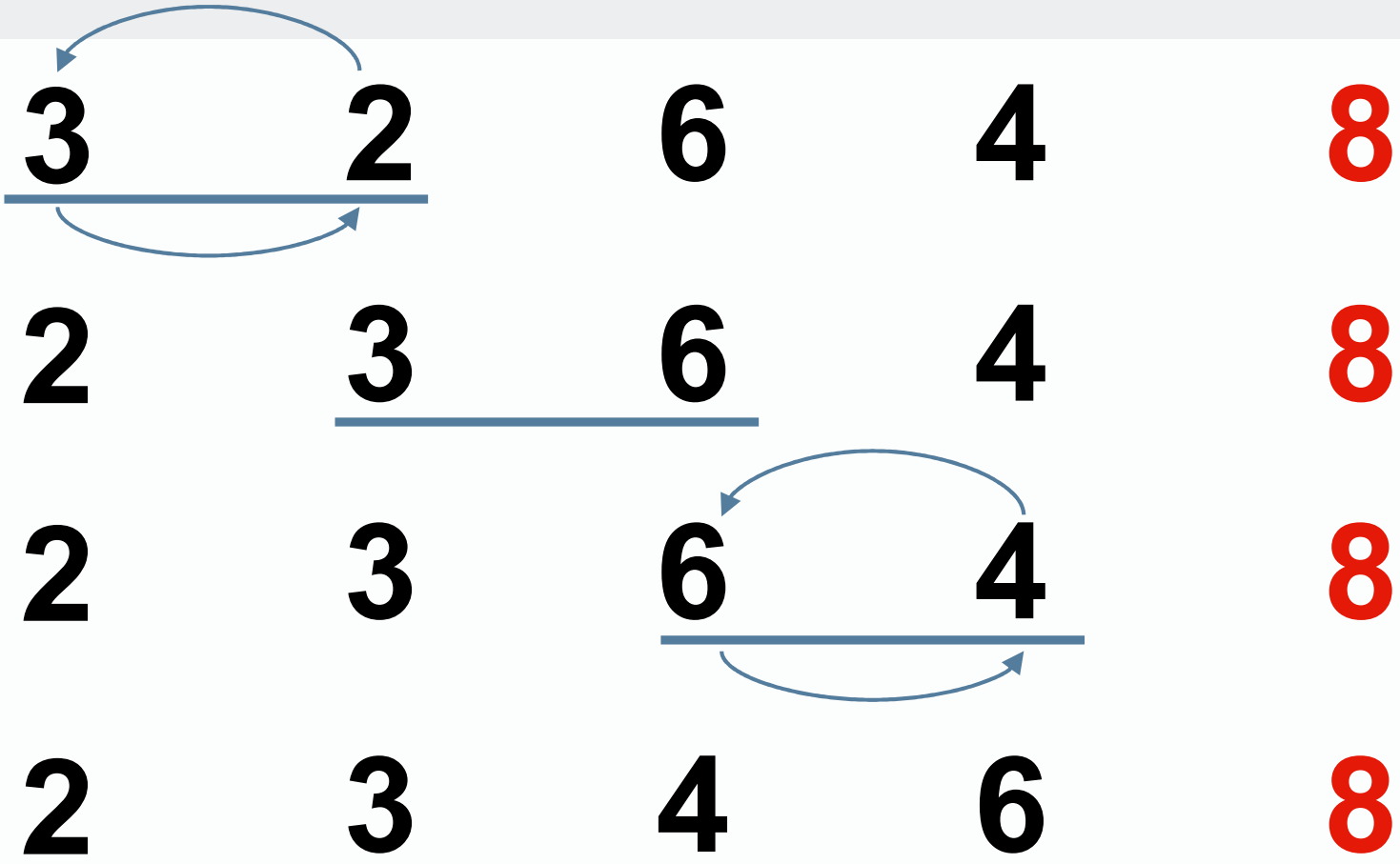
时间复杂度：$O(n^2)$， 辅助空间复杂度：O(1)， 不稳定

```
for(int i = 1; i <= n-1; ++i)
{//求a[i]~a[n]的最小值
    for(int j = i+1; j <= n; ++j)
        if(a[j] < a[i])//交换起始位置的值和最小值
            swap(a[i], a[j]);
}
```

冒泡排序



冒泡排序



冒泡排序

2 3 4 6 8

2 3 4 6 8

2 3 4 6 8

冒泡排序

2 3 4 6 8

2 3 4 6 8

2 3 4 6 8

冒泡排序

冒泡排序：依次比较两个相邻的元素，如果顺序错误就把他们交换过来。每次冒泡，最大（最小）的元素会经由交换“浮”到序列顶端。多次冒泡后，即可完成排序。

时间复杂度：$O(n^2)$， 辅助空间复杂度：O(1)，稳定

```
for(int i = 1; i <= n-1; ++i)
    for(int j = 1; j <= n-i; ++j)
        if(a[j] > a[j+1])
            swap(a[j], a[j+1]);
```

冒泡排序优化

[J-2020-5]冒泡排序算法的伪代码如下：
输入：数组L, n ≥ k。输出：按非递减顺序排序的 L。
算法 BubbleSort：

```
1. FLAG ← n //标记被交换的最后元素位置
2. while FLAG > 1 do
3.   k ← FLAG - 1
4.   FLAG ← 1
5.   for j=1 to k do
6.    if L[j] > L[j+1] then do
7.      L[j] ↔ L[j+1]
8.      FLAG ← j
```

对 n 个数用以上冒泡排序算法进行排序，最少需要比较多少次？（C）。
A. n^2　B. n-2　C. n-1　D. n

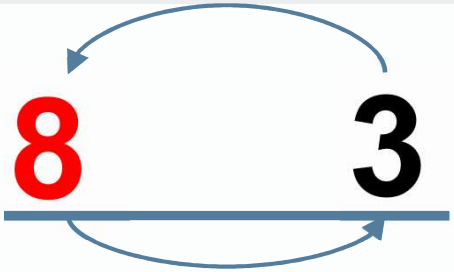
插入排序

8 3 2 6 4

8 3 2 6 4

8 3 2 6 4

插入排序

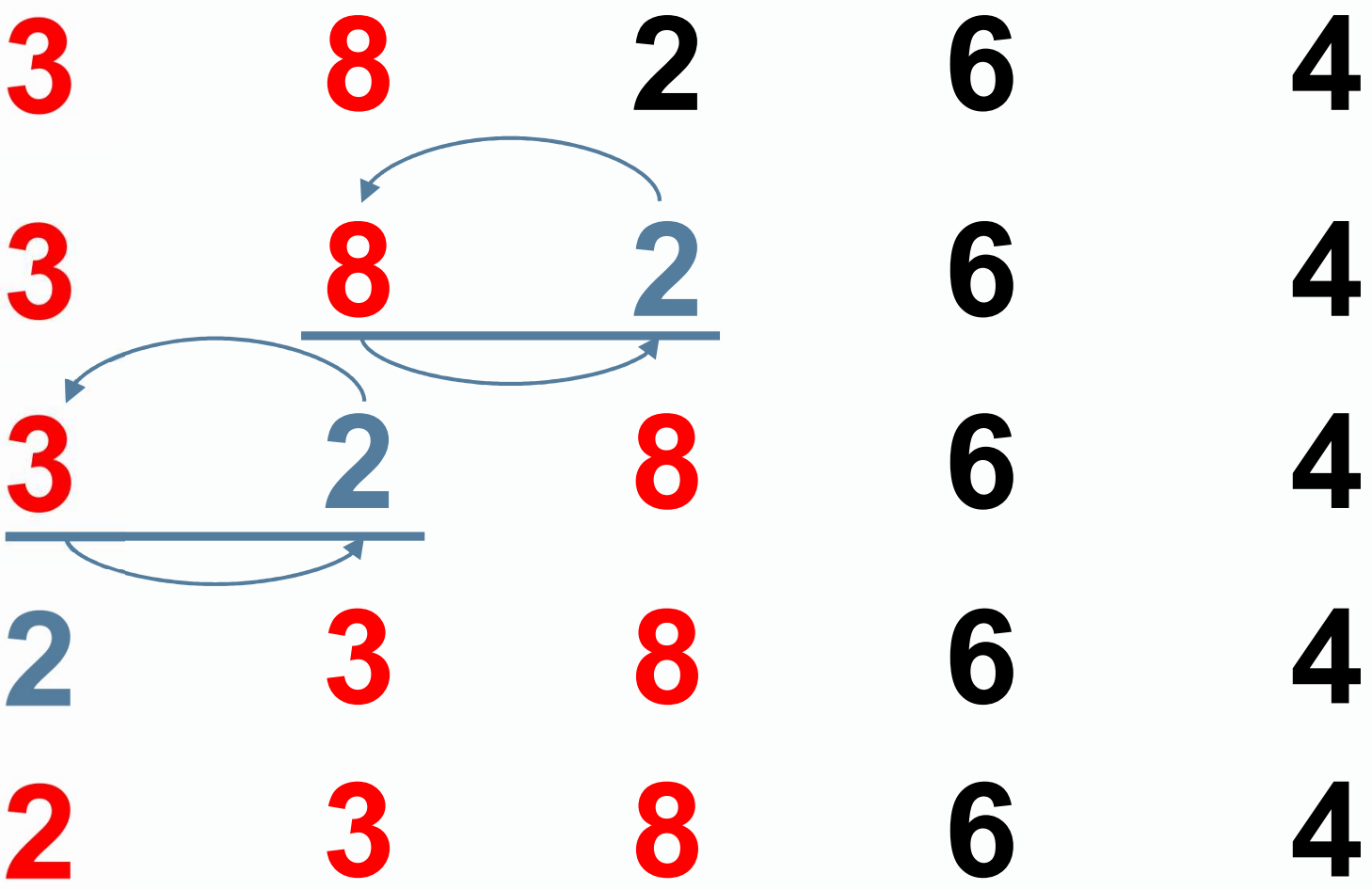


8 3 2 6 4

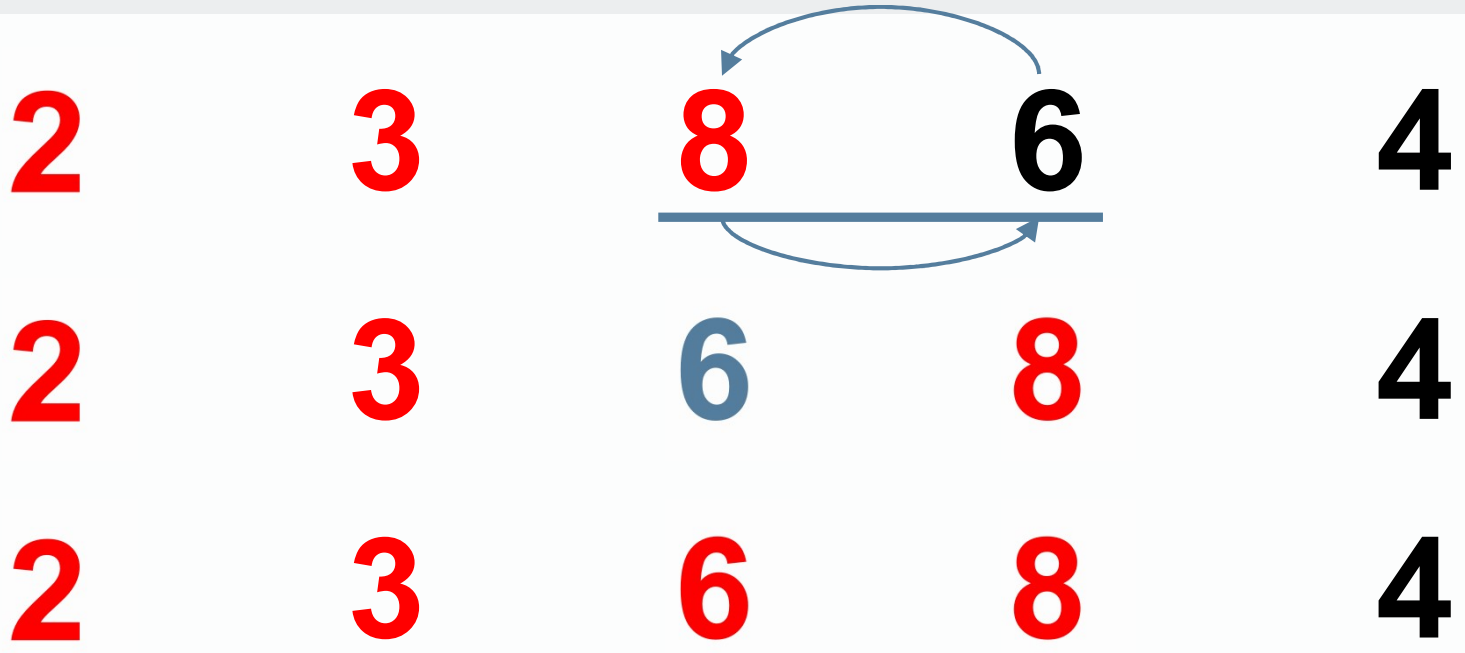
3 8 2 6 4

3 8 2 6 4

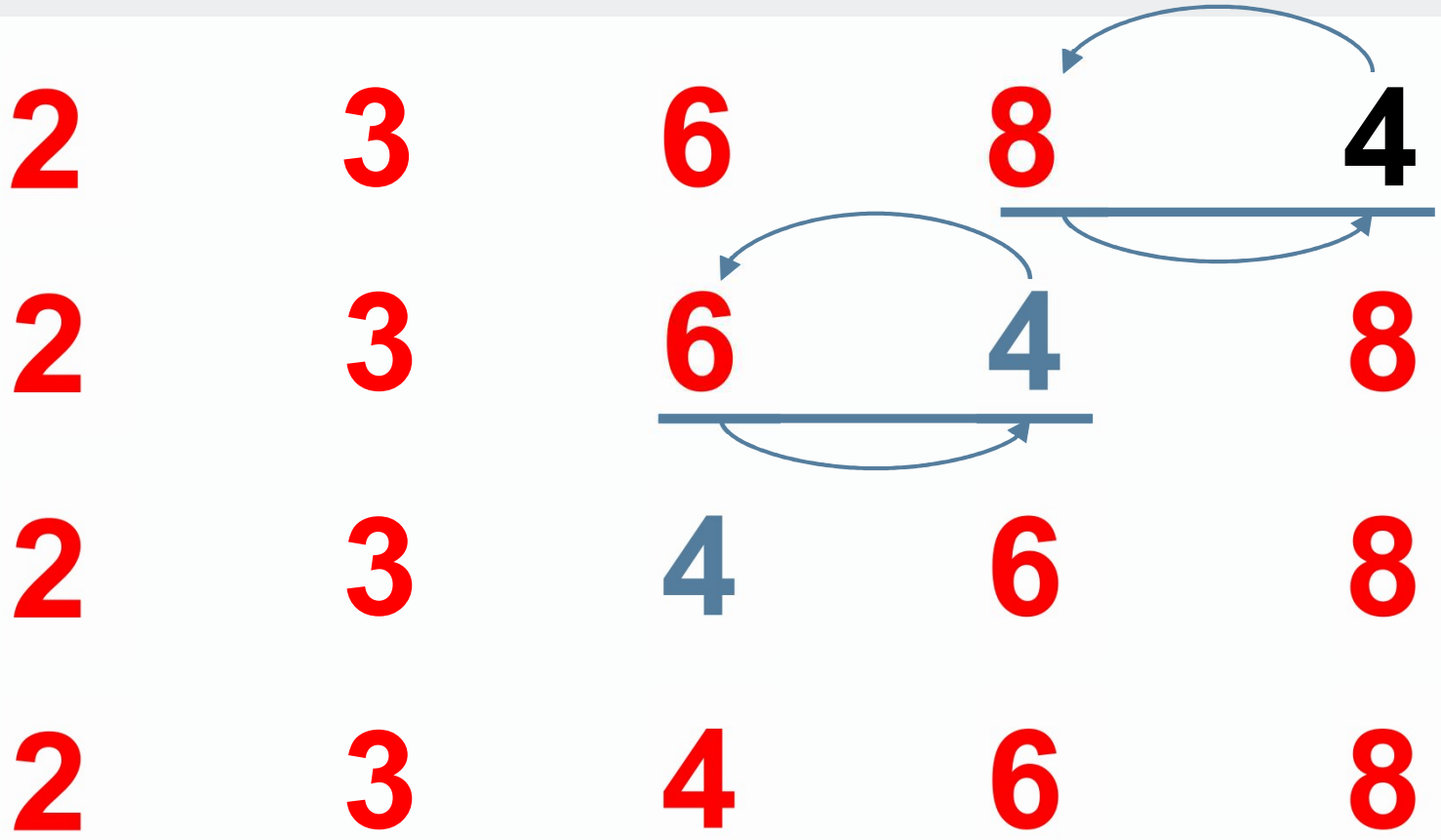
插入排序



插入排序



插入排序



插入排序

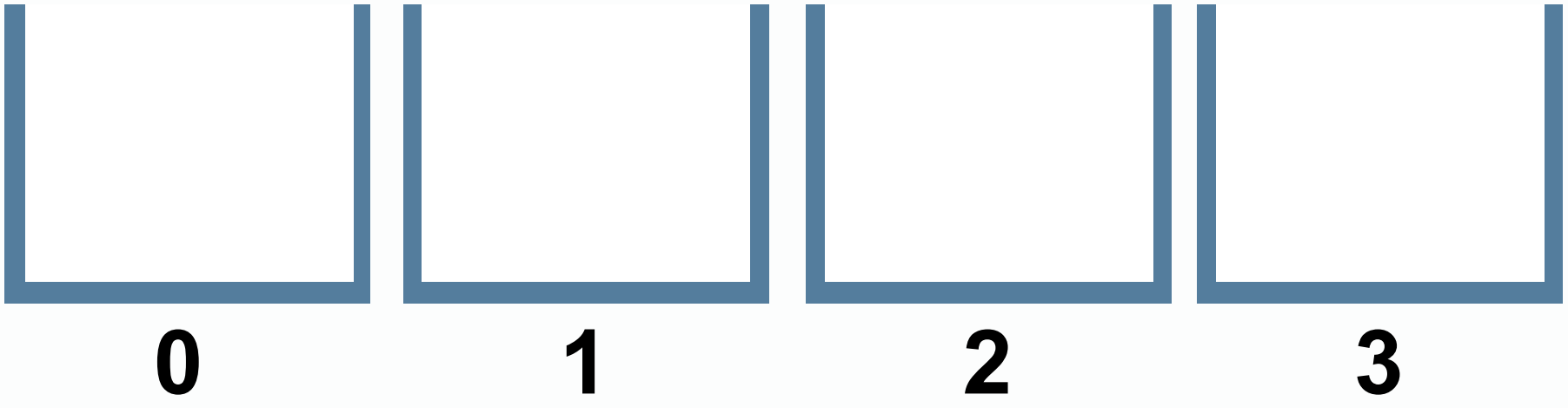
插入排序：假设前面n-1个数已经是有序的，现将第n个数插到前面已经有序的序列中，使得插入后这个序列仍然是有序的。

时间复杂度：$O(n^2)$， 辅助空间复杂度：O(1)，稳定

```
for(int i = 2; i <= n; ++i)//插入第i个数字
    for(int j = i; j > 1; j--)//j指向待插入数字
        if(a[j] < a[j-1])
            swap(a[j], a[j-1]);
        else
            break;
```

桶排序

8 11 7 23 35 32 13



桶排序

桶排序：将数据分到有限数量的有序的桶里，每个桶内分别排序，最后将各个桶中的数据合并。

数据分配到桶：散列过程

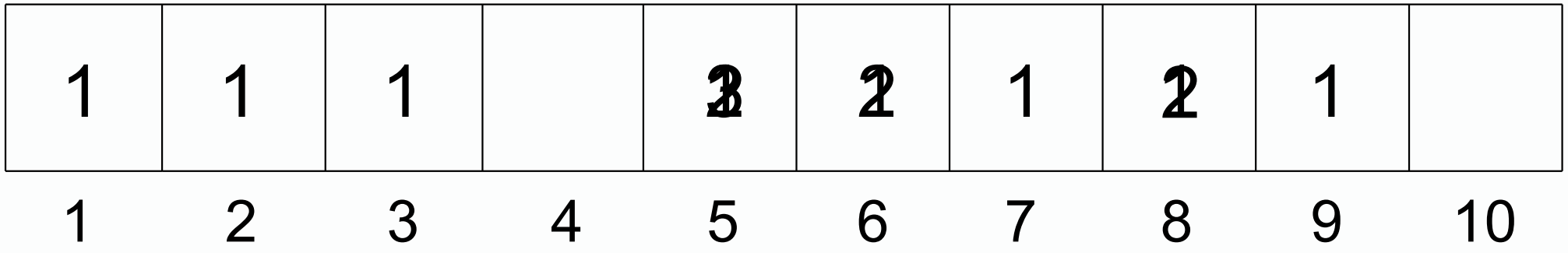


计数排序

1. 构造计数数组

1, 5, 8, 2, 9, 7, 6, 3, 5, 6, 8, 5





计数排序

2. 输出排序

1	1	1		3	2	1	2	1	
1	2	3	4	5	6	7	8	9	10

1 2 3 5 5 5 6 6 7 8 8 9

计数排序

计数排序：是一种特殊的桶排序。桶的个数是可能出现的数据种类数。

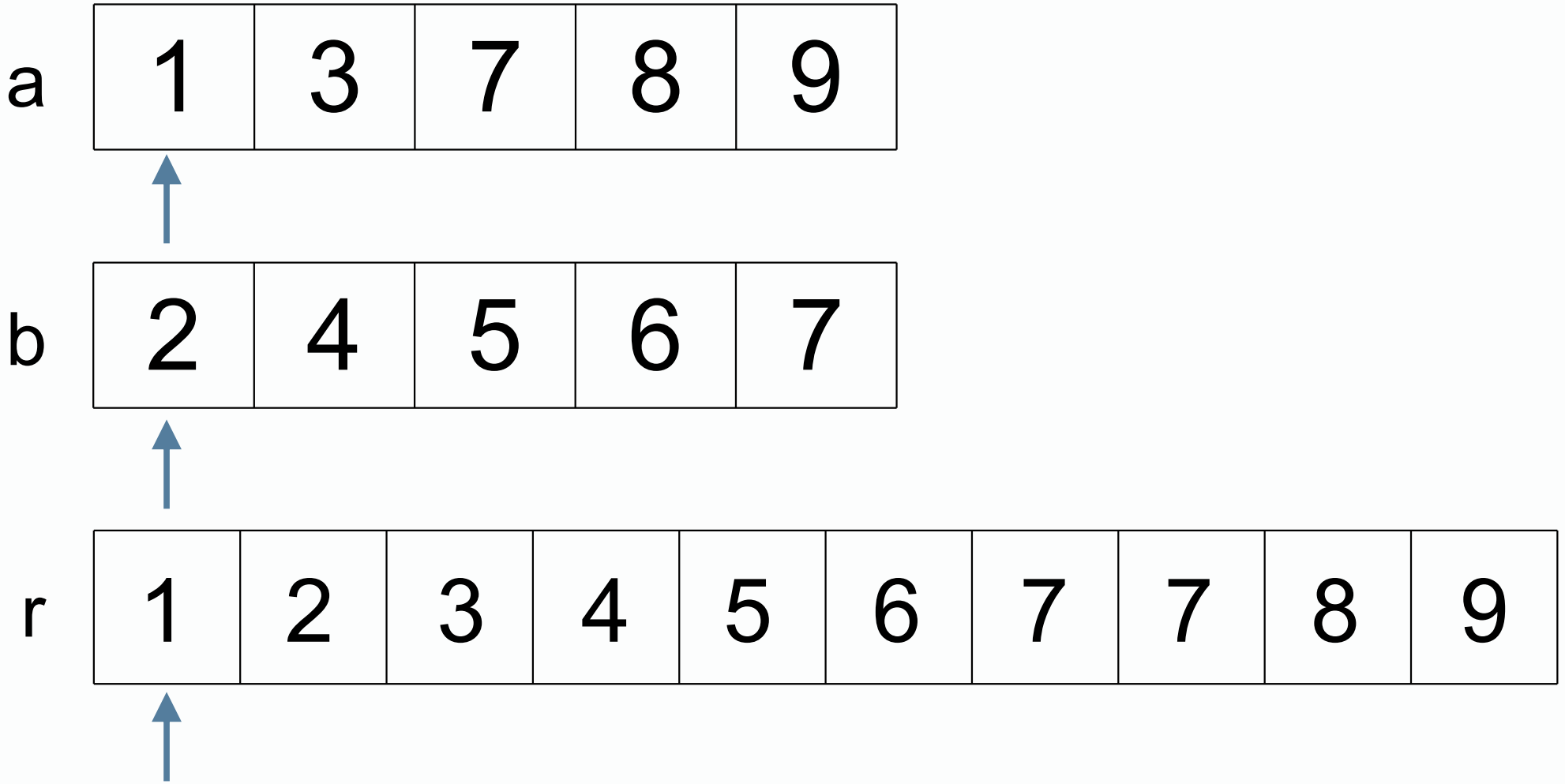
计数数组：数组下标是桶的编号，通常为数据值，数组的值表示数据个数。

适用条件：待排序数据为整数，数据范围有限

设数据个数为n，数据范围为k

时间复杂度：O(n+k)，额外空间复杂度：O(k)，稳定

归并排序



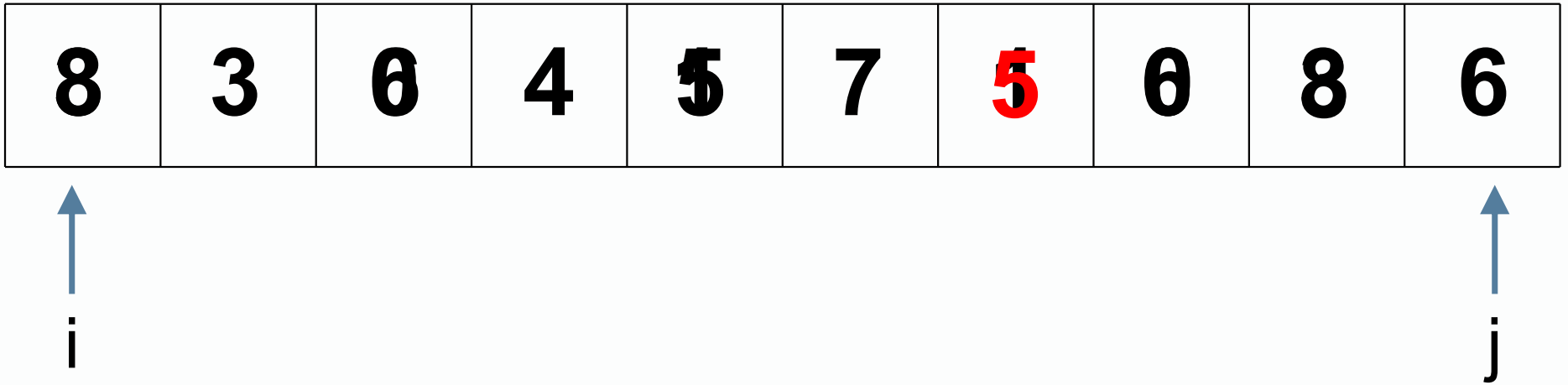
归并排序

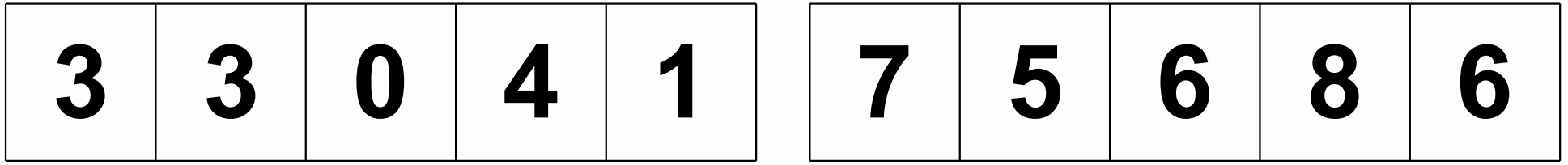
归并排序：对序列的元素进行逐层折半分组，然后从最小分组开始比较排序，逐层进行合并。

时间复杂度：O(nlogn)，辅助空间复杂度：O(n)，稳定

```
#include<bits/stdc++.h>
using namespace std;
#define N 100005
int a[N], t[N];//t:临时数组
void mergeSort(int l, int r)//a数组下标l到r进行归并排序
{
    if(l >= r)
        return;
    int mid = (l + r)/2;
    mergeSort(l, mid);
    mergeSort(mid+1, r);
    int ti = l, li = l, ri = mid+1;//li第一个段数组中的下标 ri第二段数组中的下标 ti临时数组中的下标
    while(li <= mid && ri <= r)
    {
        if(a[li] < a[ri])
            t[ti++] = a[li++];
        else
            t[ti++] = a[ri++];
    }
    while(li <= mid)//如果其中一段数组都填入t了，那么将另一段数组剩下的所有数字都填入t
        t[ti++] = a[li++];
    while(ri <= r)
        t[ti++] = a[ri++];
    for(int i = l; i <= r; ++i)
        a[i] = t[i];
}
int main()
{
    int n;
    cin >> n;
    for(int i = 1; i <= n; ++i)
        cin >> a[i];
    mergeSort(1, n);
    for(int i = 1; i <= n; ++i)
        cout << a[i] << ' ';
    return 0;
}
```

快速排序





快速排序

快速排序：每一轮挑选一个基准元素，并让比它大或小的元素移动到基准元素的两边，把数列拆解成了两个部分。而后对这两部分分别进行快速排序。

时间复杂度：O(nlogn)，辅助空间复杂度：O(logn)，不稳定

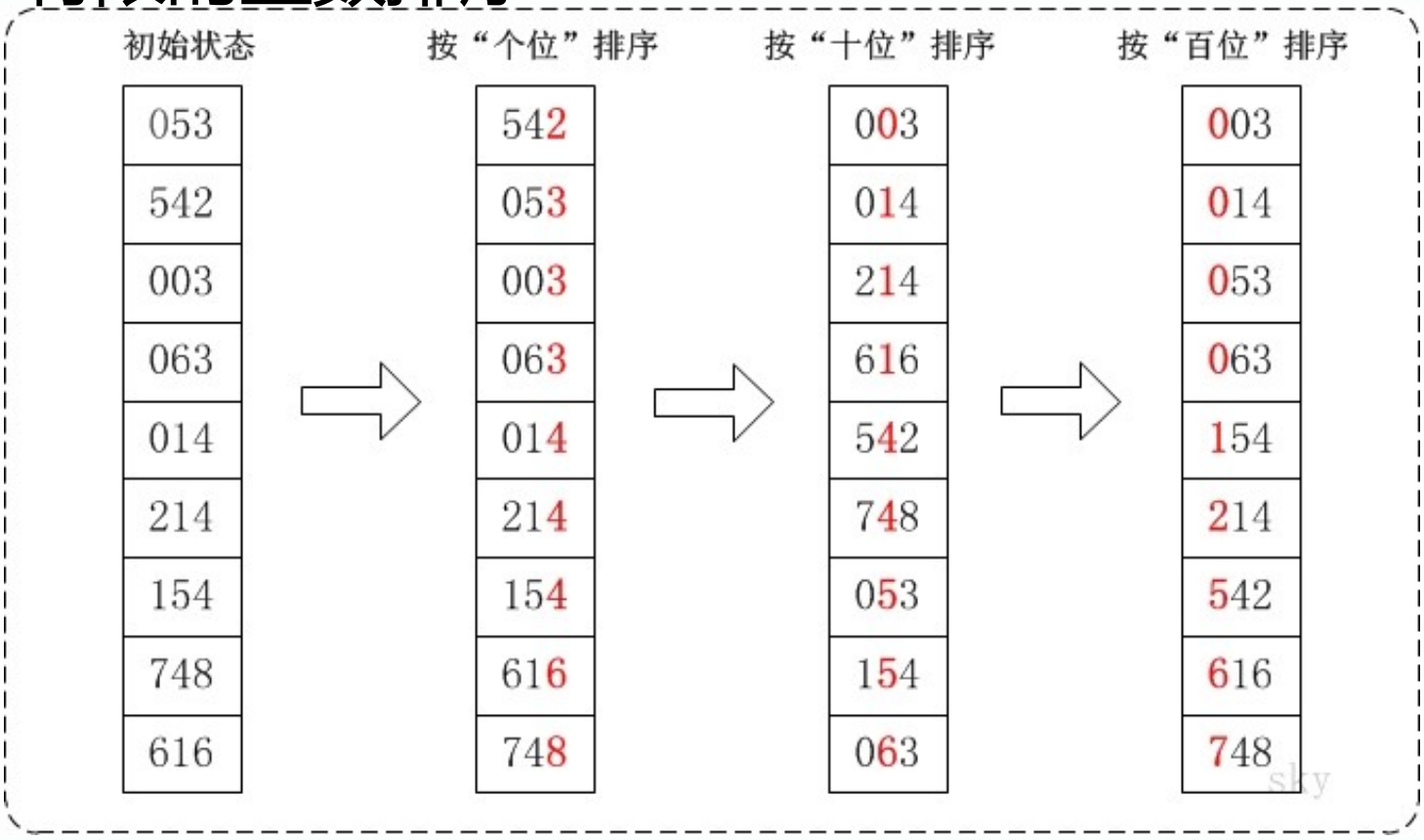
最坏时间复杂度：**$O(n^2)$**

在原序列有序时，快速排序的复杂度退化为$O(n^2)$。为避免该情况发生，可以随机选择范围内的任意数字作为基准元素。



基数排序

基数排序：一种整数排序算法，相当于多趟桶排序，适合位数有限的整数排序



时间复杂度：O(d(n+r))

辅助空间复杂度：O(n+r)

（d是数字位数，n是数字个数，r是基数）

稳定

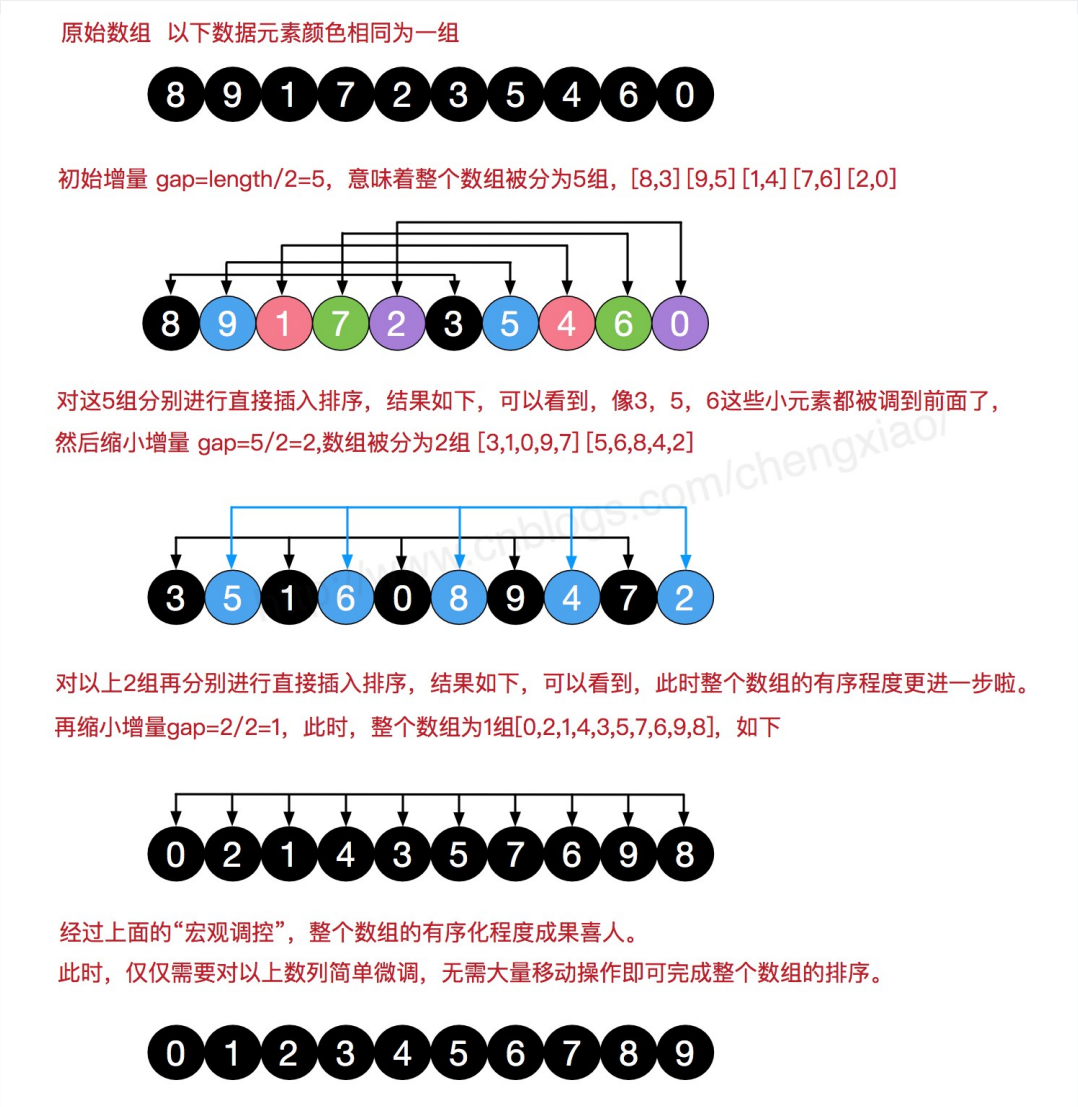
希尔排序

希尔排序：实质上是一种分组插入方法，它通过比较相距一定间隔的元素来进行，各趟比较所用的距离随着算法的进行而减小，直到只比较相邻元素的最后一趟排序为止。

时间复杂度：$O(n^{1.3})$

辅助空间复杂度：O(1)

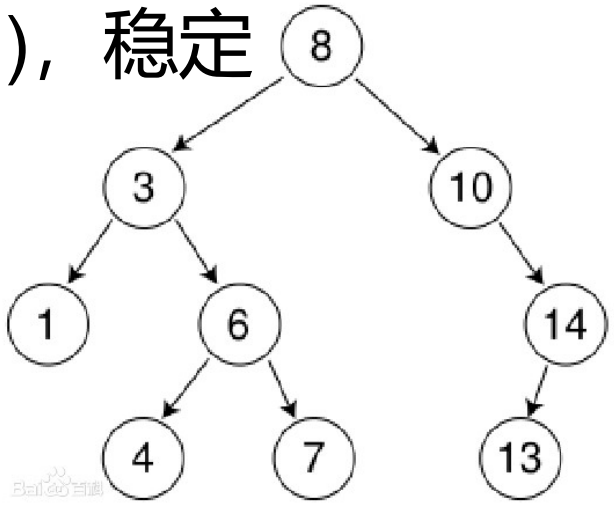
不稳定



二叉树排序

左子树上所有结点的值都小于根结点的值，右子树上所有结点的值都大小根结点的值；对二叉排序树做中序遍历，即为有序序列。

时间复杂度 O(nlogn)，辅助空间复杂度 O(n)，稳定



堆排序

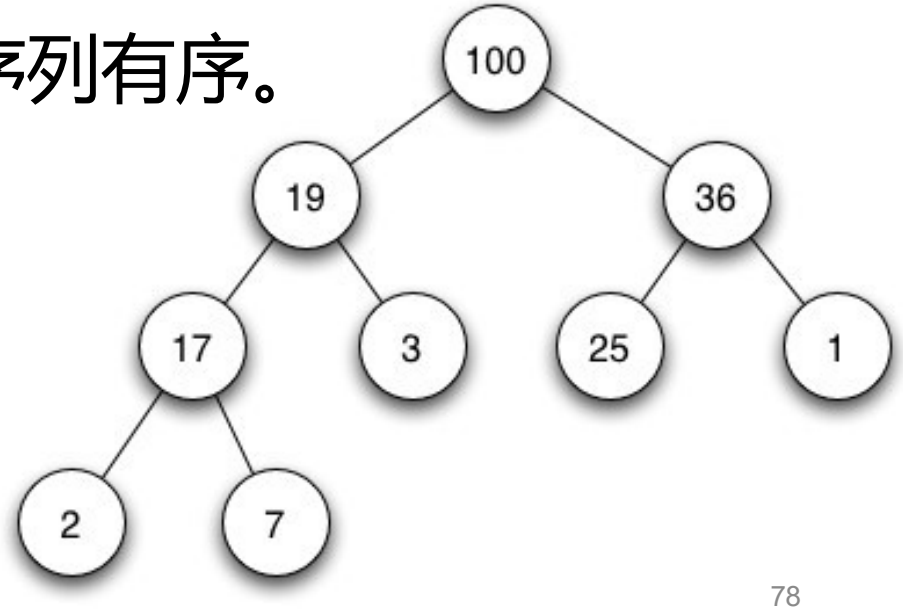
堆是一棵顺序存储的完全二叉树。

其中每个结点的关键字都不小于其孩子结点的关键字，这样的堆称为大根堆。将堆顶元素与末尾元素交换，将最大元素"沉"到数组末端，反复执行此过程，直到序列有序。

时间复杂度 O(nlogn)

辅助空间复杂度 O(1)

不稳定



各种排序算法比较

类别	排序方法	时间复杂度			空间复杂度（辅助存储）	稳定性
		平均情况	最好情况	最坏情况		
插入排序	直接插入	$O(n^2)$	O(n)	$O(n^2)$	O(1)	稳定
	希尔排序	$O(n^{1.3})$	O(n)	$O(n^2)$	O(1)	不稳定
选择排序	直接选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	不稳定
	堆排序	O(nlogn)	O(nlogn)	O(nlogn)	O(1)	不稳定
交换排序	冒泡排序	$O(n^2)$	O(n)	$O(n^2)$	O(1)	稳定
	快速排序	O(nlogn)	O(nlogn)	$O(n^2)$	O(1)	不稳定
桶排序	计数排序	O(n+k)	O(n+k)	O(n+k)	O(k)	稳定
	基数排序	O(d(r+n))	O(d(r+n))	O(d(r+n))	O(r+n)	稳定
其它排序	归并排序	O(nlogn)	O(nlogn)	O(nlogn)	O(n)	稳定
	二叉树排序	O(nlogn)	O(nlogn)	$O(n^2)$	O(n)	稳定

sort函数

sort函数，原理是**快速排序**，不稳定，时间复杂度O(nlogn)

sort(起始元素指针，最后一个元素的指针的后一个位置, 比较函数);

```
//对数组a下标1到n进行快速排序
sort(a+1, a+1+n, cmp);
//对数组a下标0到n-1进行快速排序
sort(a, a+n, cmp);
```

sort函数

sort(指向起始元素的迭代器，指向最后一个元素后一个位置的迭代器，比较函数）；

```
vector<int> vec;
sort(vec.begin(), vec.end(), cmp);
```

stable_sort函数

stable_sort函数，原理是归并排序，稳定，时间复杂度O(nlogn)

stable_sort(起始元素指针，最后一个元素的指针的后一个位置, 比较函数);

```
//对数组a下标1到n进行归并排序
stable_sort(a+1, a+1+n, cmp);
//对数组a下标0到n-1进行归并排序
stable_sort(a, a+n, cmp);
```

比较函数

比较函数：传入两个数组中元素类型的参数，
返回第一个参数排在前面的条件。
若不写比较函数cmp，默认为升序排序。

```
bool cmp(int a, int b)
{//a < b: 升序 a > b: 降序
    return a < b;
}
```



谢谢